

UNIVERSITÀ DEGLI STUDI DI SALERNO



DIPARTIMENTO DI INFORMATICA

CORSO DI LAUREA MAGISTRALE IN INFORMATICA

PENETRATION TESTING AND ETHICAL HACKING

Static Malware Analysis Automation Framework

Framework modulare per l'analisi statica di malware

Candidato:

ANTONIO GAROFALO

Professori:

PROF. ARCANGELO CASTIGLIONE

ANNO ACCADEMICO 2024/2025

Indice

1	Introduzione	2
1.1	Contesto e motivazioni	2
1.2	Obiettivi del progetto	4
2	Tecnologie e Strumenti	5
2.1	Linguaggi e librerie	5
2.2	Framework e tool esterni	7
2.3	Tecnologie pianificate per moduli futuri	9
3	Architettura del Sistema	10
3.1	Flusso generale	10
3.2	Diagramma dei componenti	12
4	Descrizione dei Moduli	15
4.1	Modulo 1: Malware Sample Collector	16
4.2	Modulo 3: Disassembler Module	22
4.3	Modulo 4: String and Signature Extractor	28
4.4	Modulo 4: Behavioral Predictor	34
4.5	Modulo 5: Reporting Engine	36
5	Conclusioni e Sviluppi Futuri	38
5.1	Sintesi dei risultati	38
5.2	Miglioramenti possibili	39

Capitolo 1

Introduzione

1.1 Contesto e motivazioni

Negli ultimi decenni, la diffusione di Malware ha assunto proporzioni sempre più rilevanti, diventando una delle *minacce principali* alla sicurezza informatica. Dalle prime forme di virus informatici, concepiti più come esperimenti che come strumenti criminali, si è passati a campioni sofisticati progettati per finalità ben precise: **spionaggio**, **estorsione**, **sabotaggio di infrastrutture critiche** e **frodi economiche** su larga scala. La complessità del panorama delle minacce ha portato i team di sicurezza a sviluppare metodologie di analisi in grado di comprendere, contrastare e anticipare i comportamenti malevoli.

Tradizionalmente, l'approccio più diffuso è quello dell'*analisi dinamica*, ovvero l'esecuzione controllata del campione all'interno di una Sandbox, con lo scopo di osservarne direttamente il comportamento.

Questo metodo ha il vantaggio di offrire una fotografia realistica delle azioni del malware, ma presenta anche diversi limiti:

- l'esecuzione del campione, per quanto in ambienti isolati, comporta **rischi** legati a possibili evasioni della sandbox;
- i campioni moderni integrano tecniche di *anti-analysis*, come il rilevamento della virtualizzazione o l'uso di temporizzazioni ritardate;
- i tempi e le risorse necessarie per eseguire analisi dinamiche approfondite risultano **elevati**.

A fronte di tali problematiche, le tecniche di *analisi statica* hanno guadagnato un ruolo sempre più centrale. Esse permettono di ispezionare un file binario senza la necessità di eseguirlo, analizzandone la struttura interna, le stringhe presenti, le chiamate a funzioni Application Programming Interface (API) e altri artefatti rilevanti. Un'analisi statica, se opportunamente automatizzata, consente di **accelerare le fasi iniziali di triage**, riducendo i costi e aumentando l'efficienza del lavoro degli analisti.

1.2 Obiettivi del progetto

Il progetto sviluppato in questa tesi nasce dall'esigenza di dotare i team di sicurezza di uno strumento capace di **automatizzare l'analisi statica** dei campioni malevoli. L'obiettivo è la realizzazione di un **framework modulare**, in grado di coprire l'intero flusso di lavoro dell'analisi, dalla raccolta del campione fino alla generazione di un report finale.

In particolare, il framework si pone i seguenti traguardi principali:

1. **Automatizzare la raccolta dei campioni:** integrando fonti eterogenee come feed pubblici, honeypot e upload manuali.
 2. **Effettuare il disassemblaggio dei binari:** tramite strumenti open source e commerciali, ottenendo rappresentazioni assembly e pseudocodice.
 3. **Estrarre stringhe e firme:** individuando indicatori di compromissione (Indicator of Compromise (IOC)) come URL, indirizzi IP, chiavi di registro, funzioni API sensibili.
 4. **Prevedere i comportamenti malevoli:** grazie a modelli di Machine Learning (ML) addestrati su dataset di malware noti, stimando capacità come *esfiltrazione di dati*, *ransomware*, *keylogging* o attività di comando e controllo.
 5. **Produrre report chiari e strutturati:** contenenti metadati, IOC, predizioni e un livello di rischio sintetico, pensati per supportare analisti e team di incident response.
-

Capitolo 2

Tecnologie e Strumenti

2.1 Linguaggi e librerie

Il linguaggio scelto per lo sviluppo del framework è stato **Python**, grazie alla sua versatilità, alla vasta disponibilità di librerie orientate alla sicurezza informatica e alla facilità di integrazione con strumenti esterni. Python ha permesso di sviluppare un sistema modulare, facilmente estendibile e adatto ad applicazioni di ricerca e sperimentazione.

Tra le librerie più rilevanti utilizzate troviamo:

- **FastAPI**: framework moderno per la creazione di interfacce Application Programming Interface (API). Ha permesso di esporre i servizi del framework tramite endpoint REST, rendendolo utilizzabile anche da applicazioni esterne o integrabile in pipeline di analisi più complesse.
- **FLARE Obfuscated String Solver (FLOSS)**: utilizzato per l'estrazione di stringhe offuscate dai binari, FLARE Obfuscated String Solver (FLOSS), è risultato particolarmente utile contro campioni che adottano tecniche di offuscamento per mascherare informazioni critiche come domini o comandi remoti.

- **YARA**: strumento open source per la classificazione di Malware basata su regole. L'integrazione con Python ha consentito di applicare regole di matching a interi set di campioni.
- **Regex (re)**: fondamentali per l'individuazione automatica di Indicator of Compromise (IOC) all'interno delle stringhe, permettono di rilevare pattern quali indirizzi IP, URL e chiavi di registro.

Docker

Per garantire sicurezza e portabilità, i diversi componenti del framework sono stati pensati per essere eseguiti in ambienti isolati tramite **Docker**.

L'uso di Docker offre diversi vantaggi:

- **Isolamento**: ogni componente può essere eseguito in un container dedicato, riducendo i rischi derivanti dall'interazione con campioni malevoli.
- **Riproducibilità**: l'ambiente di esecuzione è definito in maniera deterministica, evitando problemi di compatibilità tra librerie o versioni.
- **Portabilità**: il framework può essere distribuito ed eseguito in modo coerente su differenti sistemi operativi e infrastrutture.
- **Scalabilità**: i container possono essere orchestrati per analizzare più campioni in parallelo, aumentando le performance complessive.

L'impiego di Docker risponde quindi sia a esigenze di sicurezza, sia alla necessità di facilitare il deployment del framework in contesti reali.

2.2 Framework e tool esterni

Per supportare le fasi di disassemblaggio e arricchimento delle analisi, sono stati utilizzati alcuni strumenti esterni di grande diffusione nella comunità di sicurezza. In particolare:

- **Ghidra**: disassemblatore open source sviluppato dalla NSA. La modalità *headless* ha permesso di integrare Ghidra in flussi completamente automatizzati, senza necessità di interazione grafica.
- **Radare2**: framework di reverse engineering controllabile via `r2pipe`. È stato utilizzato per ottenere disassemblaggi complementari a quelli di Ghidra e per estrarre ulteriori metadati dai binari.
- **VirusShare**: repository di campioni malevoli, impiegato per alimentare il framework con file aggiornati.
- **MalwareBazaar**: piattaforma di abuse.ch che fornisce campioni malware costantemente aggiornati e classificati.
- **VirusTotal**: piattaforma di analisi collaborativa che combina decine di motori antivirus. L'integrazione via API ha permesso di arricchire le analisi con informazioni già note nella comunità di sicurezza.

Il framework utilizza le API messe a disposizione da servizi esterni come VirusTotal, VirusShare e MalwareBazaar. Queste integrazioni permettono di arricchire l'analisi con dati già noti nella comunità di sicurezza, riducendo i tempi di identificazione delle minacce.

Limiti delle API

È importante sottolineare che alcune di queste piattaforme offrono funzionalità avanzate (i.e query limit, sample download, full dataset access o analisi più dettagliate) riservate ad account **premium**. Nella realizzazione del progetto, sono state utilizzate le funzionalità disponibili nelle versioni *free*, comunque sufficienti per il contesto di sperimentazione e prototipazione del framework.

In prospettiva, l'adozione di piani premium consentirebbe di aumentare la copertura delle analisi e di accedere a dataset più ampi, rendendo il sistema ancora più efficace per attività di Threat Hunting e Incident Response.

Un esempio significativo è rappresentato dalla funzionalità di **Live Malware Analysis** offerta da VirusTotal, che consente di applicare regole YARA in tempo reale su campioni appena caricati. Grazie agli aggiornamenti costanti della piattaforma, gli analisti possono ricevere notifiche immediate quando un nuovo file corrisponde a specifiche regole di interesse, migliorando notevolmente la rapidità e la precisione nelle attività di rilevamento.

2.3 Tecnologie pianificate per moduli futuri

Il framework è stato concepito con un'architettura modulare, che rende possibile estendere facilmente le funzionalità. Due moduli, tuttavia, non sono stati ancora implementati: il **Modulo 4 (Behavioral Predictor)** e il **Modulo 5 (Reporting Engine)**.

Per la loro realizzazione sono già state identificate le seguenti tecnologie:

- **TensorFlow**: framework open source di ML e deep learning, sviluppato da Google, adatto a reti neurali complesse.
- **scikit-learn**: libreria Python per l'apprendimento automatico, utilizzata per modelli di classificazione leggeri e veloci.
- **Keras**: libreria ad alto livello integrata in TensorFlow, che semplifica la costruzione e l'addestramento di modelli neurali.
- **Jinja2** e **WeasyPrint**: strumenti che consentiranno di generare report strutturati in HTML e PDF, migliorando la fruibilità dei risultati da parte degli analisti.
- **STIX/TAXII** (documentazione ufficiale): standard per la condivisione di informazioni di Threat Intelligence, fondamentali per l'interoperabilità del framework con altre piattaforme e sistemi di sicurezza.

L'integrazione di queste tecnologie permetterà di trasformare il framework da semplice strumento di triage ad una piattaforma completa di analisi e condivisione delle minacce, ampliando notevolmente le sue potenzialità operative.

Capitolo 3

Architettura del Sistema

3.1 Flusso generale

Il framework è stato progettato con un'architettura modulare che consente di analizzare un campione malevolo seguendo un flusso ben definito. L'idea alla base è quella di passare da un input grezzo (il binario sospetto) ad un output strutturato (il report finale), attraverso una sequenza di trasformazioni intermedie.

Il flusso generale può essere riassunto nei seguenti passaggi principali:

1. **Input:** un campione binario (eseguibile Windows, APK Android, ELF Linux, ecc.) viene fornito al sistema.
2. **Malware Sample Collector:** il campione viene acquisito, catalogato e arricchito con metadati (hash, data di acquisizione, tipologia).
3. **Disassembler Module:** il file viene disassemblato, ricavando l'assembly del codice eseguibile e le informazioni sulle sezioni principali (.text, .data).
4. **String and Signature Extractor:** vengono estratte stringhe leggibili e offuscate, ricercati pattern tramite regex e applicate regole YARA per identificare famiglie conosciute.

5. **Behavioral Predictor:** a partire dalle feature statiche raccolte, un classificatore di ML stima i comportamenti potenzialmente malevoli (es. ransomware, keylogging, C2 beaconing).
6. **Reporting Engine:** tutte le informazioni raccolte vengono organizzate in un report strutturato, che include indicatori di compromissione, predizioni e livello di rischio.

Il processo è interamente automatizzato e pensato per un utilizzo in scenari di Triage rapido, fornendo all'analista una base solida per successivi approfondimenti con tecniche dinamiche o manuali.

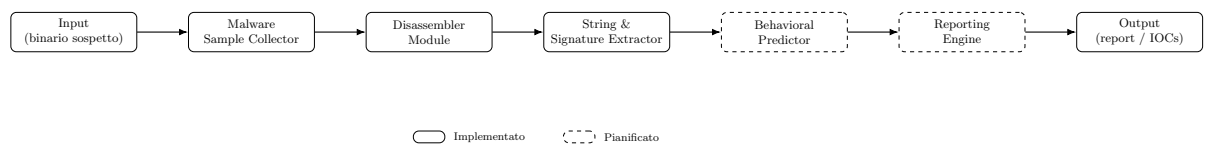


Figura 3.1: Main Flow del Sistema

3.2 Diagramma dei componenti

L'architettura del sistema è suddivisa in moduli specializzati, ciascuno con un ruolo ben definito. Questa modularità garantisce scalabilità, manutenibilità e possibilità di sostituire o aggiornare i singoli moduli senza compromettere l'intero sistema.

I componenti principali sono:

- **Malware Sample Collector:** gestisce la raccolta dei campioni da fonti esterne (es. VirusShare, MalwareBazaar, VirusTotal) e ne cura la catalogazione.
 - **Disassembler Module:** integra strumenti come Ghidra e Radare2 per tradurre il binario in assembly.
 - **String and Signature Extractor:** estrae stringhe, applica regex e regole YARA per rilevare IOC e possibili firme malware.
 - **Behavioral Predictor:** modulo previsto ma non ancora implementato, che sfrutterà tecniche di ML (scikit-learn, TensorFlow, Keras) per stimare il comportamento del campione.
 - **Reporting Engine:** modulo previsto ma non ancora implementato, che permetterà di generare report strutturati tramite Jinja2, WeasyPrint e standard STIX/TAXII.
-

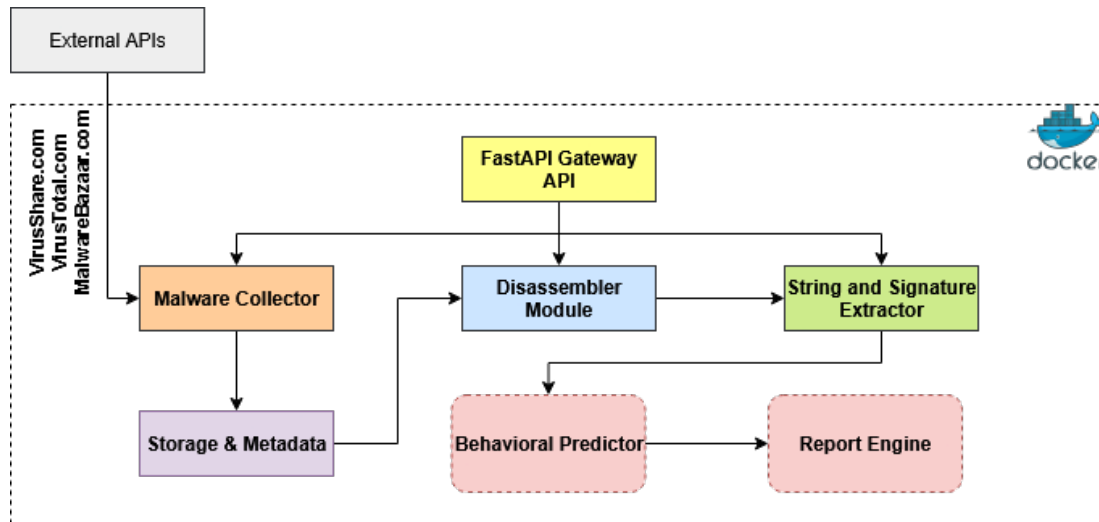


Figura 3.2: Main Flow del Sistema

Il diagramma dei componenti in Figura 3.2 illustra le relazioni tra i diversi moduli e il flusso dei dati all'interno del sistema.

All'interno del container Docker la pipeline si avvia dal **FastAPI Gateway**, che implementa le API di base per interagire con i tre moduli attualmente disponibili:

- **Malware Collector**
- **Disassembler Module**
- **String and Signature Extractor**

Lo **storage** dei campioni è realizzato in un ambiente sicuro ed isolato, configurato in modalità *sola lettura* per i moduli del framework, al fine di prevenire qualsiasi esecuzione involontaria dei file malevoli. I campioni vengono acquisiti tramite le API esterne e, attraverso il gateway del *Malware Collector*, sono importati in maniera sicura, classificati mediante **tag** descrittivi e corredati da un file **.json** contenente i metadati iniziali. Tali metadati possono successivamente essere arricchiti dall'utente attraverso ricerche incrociate o l'inserimento di ulteriori informazioni, incrementando così il valore informativo del repository.

I moduli successivi producono differenti output parziali — *metadati*, *risultati di disassemblaggio* e *stringhe/firme estratte* — che rappresentano l’input per il **Behavioral Predictor**. Quest’ultimo, una volta implementato, sarà in grado di stimare i comportamenti potenzialmente malevoli del campione. Infine, tramite il **Reporting Engine**, tutte le informazioni raccolte saranno aggregate e strutturate in un report finale, destinato sia agli analisti di sicurezza sia a sistemi esterni di supporto alle decisioni.

Nota Nella build finale saranno implementati anche il modulo 4 (**Behavioral Predictor**) e il modulo 5 (**Report Engine**) per ora programmati nei futuri sviluppi che vedremo poi.

Capitolo 4

Descrizione dei Moduli

Come evidenziato nei capitoli precedenti, il framework è stato concepito come un insieme di moduli indipendenti e cooperanti, ognuno con uno scopo ben preciso all'interno della pipeline. Questa modularità garantisce non solo una migliore manutenibilità, ma anche la possibilità di aggiornare o sostituire singoli componenti senza dover intervenire sull'intero sistema.

Nei paragrafi che seguono viene fornita una descrizione dettagliata dei cinque moduli previsti. I primi tre (*Malware Sample Collector*, *Disassembler Module* e *String and Signature Extractor*) sono già stati implementati e integrati nel framework, mentre il *Behavioral Predictor* e il *Reporting Engine* rappresentano sviluppi futuri, già delineati a livello progettuale.

4.1 Modulo 1: Malware Sample Collector

Il **Malware Sample Collector** rappresenta il punto di ingresso della pipeline di analisi ed è stato progettato per garantire la raccolta e la gestione sicura dei campioni malevoli. Questo modulo si occupa di scaricare i campioni da fonti esterne, catalogarli e predisporli per le fasi successive di disassemblaggio ed estrazione, preservando al contempo la sicurezza dell'ambiente di esecuzione.

Architettura e funzionalità

Per motivi operativi molte feed forniscono i campioni confezionati in archivi compressi (ad es. file `.zip`) protetti da password; nello scenario pratico considerato i pacchetti scaricati sono cifrati con la password standard `"infected"`.

Il modulo è quindi predisposto per gestire in modo automatizzato e sicuro questi archivi, seguendo un percorso operativo caratterizzato da verifiche e controlli stretti:

1. **Configurazione:** aggiornando il file `config.py` è possibile avere un setup personalizzabile di varie impostazioni tra cui anche le API Keys.
2. **Download dell'archivio:** il file `.zip` viene scaricato dalla fonte esterna (VirusShare, MalwareBazaar, VirusTotal) e memorizzato temporaneamente nella cartella `download/`, all'interno dell'ambiente isolato eseguito in Docker.
3. **Verifica preliminare:** prima di qualsiasi operazione di estrazione si eseguono controlli di integrità (es. verifica degli hash quando disponibili) e semplici controlli euristici per ridurre il rischio di archivi malevoli o di tipo *zip bomb*.

4. **Estrazione sicura e controllata:** l'estrazione avviene esclusivamente in ambiente isolato e con parametri restrittivi:
 - viene usata la password nota "**infected**" per decomprimere i contenuti;
 - sono applicati limiti su numero di file estratti, dimensione massima complessiva e controllo dei nomi dei file (sanitizzazione) per prevenire path traversal;
 - il processo di estrazione è eseguito con strumenti che permettono di interrompere l'operazione in caso di anomalie (es. crescita eccessiva dei file estratti).
 5. **Analisi post-estrazione e generazione metadati preliminari:** una volta estratti i file, per ogni campione viene calcolato un set minimo di metadati (hash MD5/SHA256, formato, dimensione, timestamp). Questi dati vengono salvati in un file `.json` associato e rappresentano il primo livello di **metadati preliminari**.
 6. **Trasferimento in storage di riferimento:** dopo le verifiche, i file validati sono spostati nella cartella `samples/`, che è esposta ai moduli successivi in modalità *sola lettura*. In questo modo i moduli a valle possono accedere ai dati senza possibilità di alterare i file originali.
 7. **Interazione utente e arricchimento metadati:** tramite le API offerte dal **FastAPI Gateway** l'utente può:
 - caricare manualmente archivi o singoli campioni (anch'essi sottoposti alle stesse verifiche di sicurezza);
 - interrogare e aggiornare i metadati (es. eseguire query su VirusTotal per aggiornare risultati di scansione, aggiungere tag, note o risultati di analisi manuali).
-

Questa procedura garantisce che l'intera gestione degli archivi protetti da password avvenga in modo riproducibile e sicuro, limitando l'esposizione del sistema host e fornendo all'analista un repository coerente, corredato di metadati preliminari e pronto per le fasi successive di disassemblaggio e estrazione.

Sono previste misure aggiuntive operative, quali il controllo delle dimensioni massime per estrazione, blacklist/whitelist dei tipi di file estratti e timeout per le operazioni di decompressione, per minimizzare ulteriormente i rischi operativi.

Interazione con le APIs

Il modulo sfrutta le funzionalità messe a disposizione dai provider esterni tramite API. Nella versione attuale vengono utilizzati:

- **VirusShare**: per il download di campioni grezzi.
- **MalwareBazaar**: per accedere a collezioni aggiornate di campioni malware con classificazioni aggiuntive.
- **VirusTotal**: per il reperimento di campioni e per l'arricchimento dei metadati tramite query ai motori antivirus.

Attraverso il **FastAPI Gateway**, anche l'utente può gestire direttamente i campioni, caricarne di nuovi o interrogare i metadati già esistenti, migliorando la flessibilità e l'interattività del sistema.

Endpoints

Metodo	Path	Descrizione
POST	/upload	Upload di un campione.
GET	/metadata/search	Ricerca o lista dei metadati.
POST	/metadata/update	Aggiornamento metadati via provider esterno.
POST	/providers/extract	Indicizza file scaricati dai provider.
GET	/providers/samples	Elenco file presenti nelle cartelle dei provider.
POST	/providers/query	Query ad un provider per informazioni su un sample.

Tabella 4.1: Endpoint principali del Gateway

Metodo	Path	Descrizione
GET	/virusshare/file/{hash}	Report dettagliato di un file tramite hash.
GET	/virusshare/quick/{hash}	Stato rapido di un file tramite hash.
GET	/virusshare/source/{sha256}	Informazioni di origine per file SHA-256.
GET	/virusshare/download/{hash}	Download di un campione da VirusShare.

Tabella 4.2: Endpoint per VirusShare

Metodo	Path	Descrizione
GET	/virustotal/file/{file_id}	Analisi di un file tramite ID o hash.
GET	/virustotal/url/	Report per un URL.
POST	/virustotal/scan/file	Upload di un file per la scansione.
POST	/virustotal/scan/url	Invio di un URL per la scansione.
GET	/virustotal/download/{hash}	Download di un campione da VirusTotal.

Tabella 4.3: Endpoint per VirusTotal

Metodo	Path	Descrizione
POST	/malwarebazaar/query	Query generica a MalwareBazaar.
GET	/malwarebazaar/info/{hash}	Info dettagliate di un file per hash.
GET	/malwarebazaar/recent	Ultimi campioni disponibili (limit, selector).
GET	/malwarebazaar/tag/{tag}	Campioni associati a un tag.
GET	/malwarebazaar/signature/{signature}	Campioni associati a una firma.
GET	/malwarebazaar/filetype/{filetype}	Campioni filtrati per tipo di file.
GET	/malwarebazaar/clamav/{signature}	Campioni rilevati da firma ClamAV.
GET	/malwarebazaar/imphash/{imphash}	Campioni con hash Import corrispondente.
GET	/malwarebazaar/tlsh/{tlsh}	Campioni con hash TLSH corrispondente.
GET	/malwarebazaar/telfhash/{telfhash}	Campioni con hash Telfhash corrispondente.
GET	/malwarebazaar/gimphash/{gimphash}	Campioni con hash Gimphash corrispondente.
GET	/malwarebazaar/dhash_icon/{dhash_icon}	Campioni con hash icona (dhash) corrispondente.
GET	/malwarebazaar/yarainfo/{yara_rule}	Campioni che matchano una regola YARA.
GET	/malwarebazaar/certs/issuer/{issuer_cn}	Campioni firmati da un certificato con issuer CN specifico.
GET	/malwarebazaar/certs/subject/{subject_cn}	Campioni firmati da un certificato con subject CN specifico.
GET	/malwarebazaar/certs/serial/{serial}	Campioni firmati con seriale certificato specifico.
GET	/malwarebazaar/certs/cscb	Code Signing Certificate Blocklist (CSCB).
POST	/malwarebazaar/download/{sha256}	Download di un campione tramite hash SHA-256.
POST	/malwarebazaar/submit	Invio di un campione a MalwareBazaar con metadati opzionali.

Tabella 4.4: Endpoint per MalwareBazaar

Automazione e aggiornamenti

Il Malware Sample Collector è stato concepito per funzionare in maniera continuativa e può essere configurato per aggiornare il repository di campioni:

- **su base giornaliera**, scaricando i nuovi campioni resi disponibili dai feed esterni;
- **su base oraria**, in contesti che richiedono un aggiornamento più frequente e una maggiore copertura temporale delle minacce emergenti.

Questa funzionalità consente di mantenere la collezione locale allineata con i flussi più recenti di campioni malevoli, migliorando l'efficacia del framework nelle attività di Threat Hunting e Incident Response.

Sviluppi futuri

Tra le estensioni già identificate rientrano:

- l'integrazione di **honeypot** a bassa interazione (es. Cowrie, Dionaea), per catturare campioni direttamente dall'esposizione in rete;
 - la classificazione preliminare dei campioni già in fase di raccolta, ad esempio tramite euristiche o regole YARA;
 - il supporto a ulteriori feed pubblici o privati, così da ampliare il perimetro di osservazione.
-

4.2 Modulo 3: Disassembler Module

Il **Disassembler Module** è responsabile della trasformazione dei binari in una rappresentazione assemblata leggibile, necessario per le fasi successive di estrazione di stringhe, identificazione di chiamate API e calcolo delle feature usate dai moduli di classificazione. Il progetto originario descrive chiaramente il ruolo di questo componente all'interno della pipeline: il Disassembler converte i file binari in codice assembly che viene poi passato allo String and Signature Extractor e al Behavioral Predictor.

Obiettivi funzionali

Le funzionalità principali del modulo sono:

- **Rilevamento architettura:** identificare automaticamente l'architettura del file (x86, x86_64, ARM, MIPS, etc.) e scegliere il flavour di disassembly corretto.
 - **Estrazione sezioni:** isolare e salvare le sezioni rilevanti del binario (ad es. `.text`, `.data`, tabelle di import).
 - **Disassemblaggio batch:** eseguire il disassemblaggio in modalità batch/headless per grandi volumi di file (headless Ghidra, script Radare2).
 - **Normalizzazione output:** produrre output standardizzati (assembly raw, eventuale pseudocodice, lista di chiamate API, mappe di offset) consumabili dagli altri moduli.
 - **Resilienza e sicurezza:** operare in ambiente isolato e gestire correttamente file corrotti o offuscati evitando crash di processo.
-

Strumenti integrati

Nel framework sono state adottate soluzioni open source per il disassemblaggio: in particolare Ghidra e Radare2. Questi strumenti sono stati preferiti a soluzioni commerciali come IDA Pro (che **non** è stata utilizzata nel progetto) per ragioni di licensing, automatizzabilità in modalità headless e facilità di integrazione.

Ghidra

- **Ruolo e punti di forza:** Ghidra è una suite di reverse engineering completa che offre sia un'interfaccia GUI sia una modalità *headless* adatta al disassemblaggio e all'analisi automatizzata in batch. È particolarmente indicata per analisi complesse che richiedono recovery delle funzioni, analisi dei simboli e generazione di pseudocodice.
 - **Integrazione headless:** l'esecuzione "headless" permette di lanciare script (Java/Python) su più file in modo non interattivo, utile per pipeline automatizzate. Nei casi di produzione si configura uno script che:
 1. apre il binario;
 2. imposta l'architettura e le opzioni di analisi;
 3. estrae assembly e pseudocodice;
 4. esporta risultati in formato JSON o file di testo standard.
 - **Vantaggi:** elevata qualità del disassemblaggio, supporto multi-architettura, ottime capacità di recovery sul codice offuscato.
 - **Limitazioni:** maggiore overhead in termini di memoria/CPU rispetto a strumenti più leggeri; scripting richiede attenzione alle API di Ghidra e alla configurazione dell'ambiente headless.
-

Radare2

- **Ruolo e punti di forza:** Radare2 è un framework altamente scriptabile, leggero e adatto per operazioni batch ripetitive. L'interfaccia 'r2pipe' consente di pilotare Radare2 da Python (o altri linguaggi), eseguendo comandi di disassembly e ottenendo output parsabile.
- **Esempio d'uso (integrazione Python / r2pipe):**

```
import r2pipe
r2 = r2pipe.open("sample.bin")
r2.cmd("aaa")          # analysis all
asm = r2.cmd("pd 200")  # disassemble 200 instructions
```

- **Vantaggi:** molto efficiente in risorse, ottimo per estrazioni rapide (opcode, grafi di funzione) e facilmente integrabile in pipeline Python.
- **Limitazioni:** la qualità del pseudocodice è inferiore a quella di Ghidra/IDA; richiede tuning di comandi e script per ottenere risultati completi su binari complessi.

Dettagli di implementazione

1. **Rilevamento architettura automatico:** il modulo analizza header e magic bytes del file (ad es. ELF, PE, Mach-O) per impostare automaticamente gli argomenti di disassembly.
2. **Pipeline headless:** per lotti di campioni il sistema invoca Ghidra in modalità headless per i binari che richiedono un'analisi approfondita e Radare2 per passaggi rapidi e raccolta di feature (es. opcode n-grams, chiamate import). Questa scelta ibrida bilancia qualità ed efficienza.
3. **Output standardizzato:** il Disassembler esporta:
 - file assembly plain text per ciascuna funzione/segmento;
 - elenco delle chiamate import (API calls) con offset;
 - mappa delle funzioni (start, end, nome/heuristica);
 - file JSON contenente metadati della sessione di disassembly (tool, version, opzioni).
4. **Controlli di robustezza:** ogni esecuzione viene schedulata con timeout, limiti di memoria e capture di errori. In caso di crash del disassembler il file viene marcato e messo in coda per successivi tentativi o analisi manuale.

Limiti e mitigazioni

- **Obfuscation e packing:** binari protetti tramite packer o tecniche anti-disassembly richiedono preprocessing (unpacking o emulazione) — operazioni che esulano dall'analisi statica pura e sono indicate come possibili sviluppi.
- **Falsi positivi/negativi nella ricostruzione:** la qualità del disassemblaggio dipende dalla completezza dell'analisi; per mitigare si combinano risultati di Ghidra e Radare2 e si segnalano le incongruenze per revisione manuale.

Performance, scalabilità e ottimizzazioni

Per gestire grandi volumi di campioni è prevista l'adozione di:

- **Batching e parallelizzazione:** esecuzione concorrente di istanze headless su worker container separati (orchestrati via Docker Compose / Kubernetes).
- **Prioritizzazione:** code con priorità (p.e. campioni nuovi o segnalati) per velocizzare il triage.
- **Caching dei risultati:** evitare ridisassemblaggio di file già noti (hash-based caching).

Integrazione col resto della pipeline

- Lo **String and Signature Extractor** consuma direttamente gli assembly prodotti, estraendo pattern, chiamate API e costruendo feature utili al **Behavioral Predictor**.
- I file di output (assembly, JSON) sono depositati nello storage "samples" metadata ed esposti in sola lettura ai moduli a valle per evitare modifiche accidentali.

Output

- Filtri/artefatti pronti per l'estrazione di IOC.
- File assembly e JSON di sessione utilizzabili dal Behavioral Predictor e dal Reporting Engine per generare report e indicazioni operative.

In sintesi, il *Disassembler Module* costituisce un tassello cruciale per la qualità delle feature statiche estratte: la combinazione di Ghidra (analisi profonda) e Radare2 (estrazioni rapide e scriptabili) garantisce un compromesso equilibrato tra accuratezza e scalabilità, in linea con le indicazioni progettuali del documento di riferimento.

4.3 Modulo 4: String and Signature Extractor

Il **String and Signature Extractor** costituisce il ponte tra il disassemblaggio e la fase di classificazione/scoring: a partire dai binari e dagli artefatti testuali (gli output del modulo di disassemblaggio e i metadati statici), estrae stringhe, IOC e corrispondenze YARA, producendo infine un punteggio di rischio normalizzato.

Panoramica della pipeline

Il flusso implementato è orchestrato dalla funzione `process_sample(...)`:

1. **Caricamento artefatti statici** (metadati e testi di disassemblaggio).
 2. **Estrazione stringhe** con tre approcci complementari: *strings* nativo, estrattore in Python, *FLOSS*.
 3. **Costruzione del corpus testuale** (stringhe uniche + sorgenti di disassemblaggio).
 4. **Estrazione di IOC** (URL, IP, domini, chiavi di registro, email).
 5. **Mappatura del contesto** degli IOC all'interno dei file di disassemblaggio.
 6. **Matching YARA** su binario e su corpus testuale (regole da Yara-Rules).
 7. **Scoring** euristico con normalizzazione e stima della confidenza.
 8. **Serializzazione JSON** dell'output (IOC, YARA, stringhe e scores).
-

Estrazione delle stringhe

Sono stati implementati tre meccanismi:

- **Estrattore nativo (strings):** invocato con opzioni per offset (`-t x`) e lunghezza minima; in caso di indisponibilità del comando, si cade automaticamente sull'estrattore Python.
- **Estrattore Python:** scansione del contenuto binario e raccolta di sequenze ASCII stampabili (con track dell'*offset* in esadecimale).
- **FLOSS:** integrazione via *API Python* per catturare stringhe deoffuscate; le stringhe FLOSS sono etichettate dedicate per supportare lo scoring su obfuscation.

Nota le stringhe provenienti da più fonti (nativo, Python, FLOSS, metadati statici) vengono **deduplicate** preservando il primo record “migliore” (preferendo quello con *offset* disponibile) e marcate con il campo `source` (`strings`, `strings_python`, `floss`, `static_information`).

Estrazione IOC (Regex e normalizzazioni)

Sul corpus testuale consolidato (stringhe + sorgenti disassemblati) opera un estrattore di IOC che:

- rileva **email**, **domini**, **URL**, **textbf**indirizzi IP v4 pubblici, **chiavi di registro**;
- esegue normalizzazioni (es. domini in IDNA/punycode minuscoli; canonicalizzazione degli hive Windows);
- applica filtri di validità (es. esclusione di IP privati/loopback/multicast, stripping di punteggiatura di coda).

Esempi (semplificati) di pattern impiegati:

```
EMAIL_RE = [A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}
DOMAIN_RE = (?:[A-Za-z0-9](?:[A-Za-z0-9-]{0,61}[A-Za-z0-9])?\.[A-Za-z]{2,63}
URL_RE = https?://[^\s'"\<>]+
REG_RE = (?:HKEY_[A-Z_]+|HKLM|HKCU|HKCR|HKU|HKCC)\\[^\s'"\]+
IP_RE = \b\d{1,3}(\.?\d{1,3}){3}\b
```

Per ogni IOC individuato, il modulo costruisce anche una **mappa di contesto** (*path*, numero di riga, snippet con $\pm n$ righe) per facilitare il triage.

YARA Engine e Rule Sets

Il modulo compila le regole YARA da:

- un singolo file `.yar` (caso *index*);
- una directory: se è presente `index.yar` nella radice, compila solo quello; altrimenti raccoglie ricorsivamente tutti i file `.yar` e li compila con *namespace* separati.

Le regole adottate provengono dal repository pubblico Yara-Rules, ampiamente riconosciuto nella community. Il matching avviene su:

- **binario**: `scan_file(...)` con timeout difensivo;
- **corpus testuale**: `scan_text(...)` sul buffer (stringhe+codice) per intercettare firme testuali.

Per ciascuna corrispondenza si serializzano *rule*, *namespace*, *tags*, *meta* e un sottoinsieme di *matched strings* (con *offset*, *identifier*, preview ASCII/HEX), limitando il numero massimo di stringhe per evitare output eccessivi.

Scoring euristico (basato su evidenze reali)

Lo **scoring system** combina contributi da YARA, IOC, *behavioral cues* (pattern di API nei testi), **obfuscation** (rapporto di stringhe FLOSS) e metadati statici (entropia/packer). Le principali logiche sono:

- **YARA**: base score per regole sul binario e sul testo (con *cap* sul numero conteggiato), bonus per meta che indicano *family/name* e per hit multipli sul binario.
- **IOC**: contributi lineari con pesi e *cap* separati (URL, IP, domini, registro), più bonus di **persistence** se alcune chiavi di registro note sono presenti.
- **API/behavior**: incremento se nel corpus compaiono API indicative (es. `VirtualAllocEx`, `WriteProcessMemory`, `connect/send/recv`, `RegSetValue`, `IsDebuggerPresent`, ecc.).
- **Obfuscation**: bonus se la quota di stringhe FLOSS supera soglie minime (conteggio e rapporto sul totale).
- **Statici**: piccole maggiorazioni per *entropia elevata* e *packer* rilevato.

Il punteggio grezzo viene **normalizzato** in scala 0–100 tramite una curva esponenziale “soft-cap” (rapida saturazione per evitare outlier):

$$\text{score}_{\text{norm}} = \left\lfloor 100 \cdot \left(1 - e^{-\frac{\text{score}_{\text{raw}}}{\lambda}}\right) \right\rfloor \quad \text{con } \lambda \approx 85.$$

Viene inoltre stimata una **confidence** qualitativa (*low/medium/high*) in base al punteggio normalizzato e alla *varietà* delle evidenze (numero di categorie con contributo > 0).

Limiti e mitigazioni

- **False hit regex:** mitigati con normalizzazioni, filtri IP pubblici, IDNA e stripping punteggiatura;
- **Rumore FLOSS:** ridotto con soglie su lunghezze e conteggio minimo/rapporto;
- **Regole YARA eterogenee:** serializzazione limitata delle matched strings e *timeout* su scan.

Output

L'output è un oggetto JSON con:

- `file`, `static_info` (metadati di base);
- `strings` (elenco record `{string, source, offset}`);
- `iocs` e `iocs_context` (`token`, `path`, `line`, `snippet`);
- `yara_matches_bin`, `yara_matches_text` (regola, tag, meta, matched strings truncate);
- `score`, `score_detail` (`raw_score`, `reasons`, `confidence`, `evidence_categories`).

In sintesi, lo *String and Signature Extractor* fornisce una base solida e riproducibile per l'identificazione rapida di indicatori e di firme note: combina metodi semplici (stringhe in chiaro), tecniche avanzate (FLOSS) e YARA (regole da Yara-Rules). Lo scoring, pur *euristico*, è ancorato a **evidenze concrete** (IOC reali, meta YARA, API indicative, obfuscation) e restituisce un punteggio normalizzato con stima della confidenza, utile per il Triage e per la prioritizzazione delle analisi.

4.4 Modulo 4: Behavioral Predictor

Il **Behavioral Predictor** è concepito come il componente di intelligenza artificiale del framework. La sua funzione è quella di stimare i comportamenti potenzialmente malevoli di un campione sulla base delle informazioni ottenute dai moduli precedenti (metadati, disassemblaggio, stringhe e firme).

Funzionalità previste

- Analizzare le sequenze di *opcode* e le chiamate API estratte durante il disassemblaggio.
- Identificare pattern ricorrenti associati a particolari famiglie o comportamenti di Malware.
- Utilizzare tecniche di ML per classificare i campioni in categorie comportamentali (es. *ransomware*, *keylogger*, *Remote Access Trojan*).
- Fornire un punteggio di confidenza per ogni predizione, utile per stabilire il livello di rischio.

Tecnologie pianificate

Il modulo farà uso di librerie di machine learning già identificate in fase di progettazione:

- scikit-learn, per algoritmi classici (es. Random Forest, Naive Bayes).
- TensorFlow e Keras, per modelli più complessi come LSTM e CNN, in grado di catturare relazioni sequenziali tra istruzioni o chiamate di sistema.

Output

Il risultato del modulo sarà un set di etichette comportamentali con i relativi punteggi di probabilità, che costituiranno input diretto per il *Reporting Engine*.

Ad esempio:

- Ransomware (85%)
- Keylogging (40%)
- C2 beaconing (70%)

In sintesi, il *Behavioral Predictor* è progettato come il motore di intelligenza artificiale del framework: la sua funzione è trasformare gli artefatti statici in predizioni di comportamento, fornendo un primo livello di classificazione automatica delle minacce. Sebbene non ancora implementato, la sua architettura, basata su scikit-learn, TensorFlow e Keras, permetterà di stimare in maniera probabilistica le finalità operative dei campioni (es. ransomware, keylogger, RAT). La disponibilità di tali predizioni consentirà di velocizzare il triage e migliorare la priorità nelle attività di Incident Response.

4.5 Modulo 5: Reporting Engine

Il **Reporting Engine** rappresenta il punto di aggregazione dei risultati prodotti dal framework. Il suo obiettivo è la generazione di report strutturati, che condensino in modo chiaro e leggibile i risultati delle analisi statiche e delle predizioni comportamentali.

Funzionalità previste

- Integrare i risultati dei moduli precedenti (metadati, IOCs, disassemblaggio, estrazioni, predizioni).
- Organizzare le informazioni in sezioni chiare (metadati file, IOC estratti, predizioni, livello di rischio).
- Consentire l'esportazione in diversi formati, per rispondere a esigenze diverse (es. HTML per la consultazione web, PDF per la distribuzione statica, STIX/TAXII per l'integrazione con sistemi di Threat Intelligence).

Tecnologie pianificate

- Jinja2, per la creazione di template HTML modulari e personalizzabili.
- WeasyPrint, per la conversione da HTML a PDF.
- STIX/TAXII, per la serializzazione e la condivisione dei risultati in contesti di collaborazione.

Output

Il modulo produrrà report strutturati e standardizzati, destinati sia agli analisti di sicurezza sia a sistemi esterni. Ogni report includerà:

- **Metadati del campione:** hash, dimensione, formato, data di acquisizione.
- **Indicatori di compromissione (IOC):** stringhe, URL, indirizzi IP, chiavi di registro.
- **Predizioni comportamentali:** etichette con i rispettivi punteggi di confidenza.
- **Livello di rischio:** classificato in Low, Medium, High o Critical.

In sintesi, il *Reporting Engine* rappresenterà il punto di convergenza delle analisi effettuate dai moduli precedenti. Integrando metadati, IOC, risultati di disassemblaggio e predizioni comportamentali, questo modulo produrrà report chiari, completi e standardizzati. L'impiego di Jinja2, WeasyPrint e standard come STIX/TAXII garantirà la fruibilità dei risultati sia da parte di analisti umani sia da piattaforme esterne di Threat Intelligence, consolidando il framework come strumento pratico e interoperabile.

Capitolo 5

Conclusioni e Sviluppi Futuri

5.1 Sintesi dei risultati

Il progetto presentato ha portato alla realizzazione di un **framework modulare** per l'analisi statica di Malware, capace di automatizzare molte delle attività tradizionalmente svolte manualmente dagli analisti.

Grazie all'integrazione di diversi moduli specializzati, il sistema consente di:

- raccogliere campioni da fonti eterogenee e gestirne i metadati;
- disassemblare binari tramite strumenti open source e commerciali;
- estrarre stringhe e IOC tramite regex, firme e regole YARA;
- predisporre l'infrastruttura per la predizione dei comportamenti malevoli mediante modelli di ML;
- generare report strutturati, contenenti informazioni tecniche e un livello di rischio sintetico.

Il framework rappresenta dunque un valido strumento di supporto per il Triage iniziale dei campioni sospetti, in grado di ridurre sensibilmente i tempi di analisi e di fornire un punto di partenza solido per indagini più approfondite.

5.2 Miglioramenti possibili

Nonostante i risultati ottenuti, il framework può essere ulteriormente potenziato attraverso diverse linee di sviluppo:

1. **Finalizzare i moduli mancanti:** attualmente il *Modulo 4 (Behavioral Predictor)* e il *Modulo 5 (Reporting Engine)* sono stati progettati ma non ancora realizzati. La loro implementazione rappresenta un passo fondamentale per raggiungere la piena funzionalità del framework.
2. **Integrazione con analisi dinamica:** combinare l'analisi statica con dati raccolti da Sandbox permetterebbe di validare e arricchire le predizioni.
3. **Aggiornamento continuo delle firme:** integrare feed di regole YARA e database di IOC aggiornati costantemente garantirebbe maggiore efficacia contro nuove minacce.
4. **Scalabilità e parallelizzazione:** ottimizzare i moduli di disassemblaggio ed estrazione per l'elaborazione in batch di grandi volumi di campioni.
5. **Miglioramento dei modelli di apprendimento automatico:** sperimentare reti neurali più avanzate (ad esempio LSTM o Transformers) per incrementare l'accuratezza della classificazione comportamentale.
6. **Condivisione delle informazioni di threat intelligence:** adottare formati standard come Structured Threat Information Expression (STIX)/Trusted Automated Exchange of Indicator Information (TAXII) per integrare i report con piattaforme di collaborazione fra enti e organizzazioni.
7. **Sviluppo di una interfaccia utente:** realizzare una *User Interface (UI)* intuitiva, ad esempio come applicazione web, per consentire anche ad analisti meno esperti di interagire facilmente con il framework e visualizzare i risultati delle analisi.

Questi sviluppi futuri renderebbero il framework non solo uno strumento efficace per l'analisi preliminare, ma anche una componente strategica all'interno di ecosistemi più ampi di Threat Intelligence.

Nota finale

In conclusione, il framework realizzato rappresenta un primo passo concreto verso la creazione di una piattaforma di **analisi statica automatizzata** che possa supportare i team di sicurezza nella gestione dell'elevato numero di campioni malevoli che emergono quotidianamente.

Sebbene alcune componenti, come il *Behavioral Predictor* e il *Reporting Engine*, siano ancora in fase di progettazione, l'architettura sviluppata fornisce solide basi per una futura estensione. L'introduzione di ulteriori moduli, l'adozione di tecniche di ML avanzate e la realizzazione di una *interfaccia utente* intuitiva potranno rendere questo framework non solo uno strumento utile per il triage preliminare, ma anche un elemento cardine all'interno di strategie di Threat Intelligence moderne e collaborative.

Il valore di questo lavoro risiede dunque nella sua duplice natura: da un lato **dimostrazione pratica** delle potenzialità dell'automazione nell'analisi statica, dall'altro **base solida** su cui costruire strumenti sempre più completi, affidabili e utilizzabili in contesti reali di difesa cibernetica.

Glossario

- Docker** Piattaforma open source che consente di automatizzare la distribuzione e l'esecuzione di applicazioni all'interno di container software. Garantisce isolamento, portabilità e riproducibilità degli ambienti, rendendola particolarmente utile in contesti di sicurezza informatica.
- FastAPI** Framework web moderno e performante scritto in Python, utilizzato per creare interfacce API REST e applicazioni web veloci.
- Ghidra** Suite open source sviluppata dalla NSA per il reverse engineering e il disassemblaggio di binari, utilizzata sia con interfaccia grafica che in modalità headless.
- Incident Response** Insieme di procedure e pratiche operative per identificare, contenere e mitigare un incidente di sicurezza informatica, con l'obiettivo di ripristinare la normale operatività e ridurre l'impatto dell'attacco.
- Jinja2** Motore di template per Python che consente di generare contenuti dinamici, come pagine HTML o documenti, a partire da strutture predefinite.
- Keras** Libreria open source di alto livello per reti neurali, progettata per semplificare la costruzione e l'addestramento di modelli complessi. È oggi integrata come API ufficiale di TensorFlow.
- Malware** Software malevolo progettato per compromettere la sicurezza, rubare dati, danneggiare sistemi o svolgere attività non autorizzate.
- MalwareBazaar** Progetto di *abuse.ch* che distribuisce campioni di malware raccolti e classificati, disponibili tramite interfaccia web e API.
- Radare2** Framework open source per il reverse engineering e l'analisi di binari, scriptabile tramite l'interfaccia *r2pipe*.
- Sandbox** Ambiente isolato e controllato in cui eseguire codice potenzialmente dannoso, al fine di osservarne il comportamento senza rischi per il sistema principale.
- scikit-learn** Libreria Python per il machine learning che offre algoritmi di classificazione, regressione, clustering e strumenti per la valutazione dei modelli.
- TensorFlow** Framework open source per il machine learning e il deep learning, sviluppato da Google, ampiamente utilizzato per la creazione, l'addestramento e l'esecuzione di modelli di intelligenza artificiale.
- Threat Hunting** Attività proattiva di ricerca di minacce informatiche già presenti all'interno di una rete o di un sistema, spesso non rilevate dagli strumenti di difesa tradizionali.
- Threat Intelligence** Attività di raccolta, analisi e condivisione di informazioni sulle minacce informatiche per migliorare la prevenzione e la risposta agli attacchi.
- Triage** Fase preliminare di classificazione e valutazione rapida dei campioni malevoli, con l'obiettivo di stabilire priorità di analisi e risposta.
- VirusShare** Repository pubblico di campioni malevoli che mette a disposizione dataset aggiornati di file sospetti per fini di ricerca e analisi.
- VirusTotal** Piattaforma di analisi collaborativa che consente di sottoporre file e URL a decine di motori antivirus e strumenti di sicurezza, restituendo report aggregati.
- WeasyPrint** Motore open source per la generazione di documenti PDF e immagini a partire da contenuti HTML e CSS. Utilizzato per convertire template HTML in report professionali, mantenendo formattazione e stile coerenti.
- YARA** Strumento open source per il rilevamento e la classificazione di malware basato su regole di pattern matching.

Acronimi

API	Application Programming Interface
FLOSS	FLARE Obfuscated String Solver
IOC	Indicator of Compromise
ML	Machine Learning
STIX	Structured Threat Information Expression
TAXII	Trusted Automated Exchange of Indicator Information